

From the Line Filter to the Anisotropic Stencil Filter

A Step-by-Step Mathematical Walkthrough

J.C. Vaught

1 The Big Picture (Before Any Math)

The whole point of this document is to answer one question: how did we go from the Line Filter (LF), which is slow and memory-hungry, to the Anisotropic Stencil Filter (ASF), which is fast and lean, without changing the answer at all?

The short version is this. The Line Filter does a bunch of steps (gather pixels, build a matrix, solve a system, repeat along a line, weight, sum). Every single one of those steps is a linear operation on the original pixel values. When you chain a bunch of linear operations together, the result is still just one big linear operation. That means the entire LF pipeline, no matter how complicated it looks, is secretly equivalent to “multiply each pixel in a neighborhood by some weight and add them up.” The ASF simply precomputes those weights ahead of time so the GPU only has to do the easy part: gather, multiply, sum. One operation instead of many. Same answer. Much faster.

The rest of this document walks through every step of the math that makes this work, slowly and with lots of explanation.

2 Step 1: What Are We Even Computing?

2.1 The Goal

We have a grayscale image, which is really just a grid of numbers. Each number is a brightness value. We want to find edges, the places where the brightness changes sharply.

To find edges, we need to estimate how fast the brightness is changing at every pixel. In calculus terms, we want the derivative of the image intensity. The bigger the derivative, the stronger the edge.

2.2 The Problem with Simple Approaches

The most obvious way to estimate a derivative is to look at two neighboring pixels and take the difference. This is essentially what Sobel and Prewitt filters do (with some slight averaging). The problem is that if the image is noisy, those two pixels might have random junk in their values, and the difference will be meaningless.

Our approach uses way more pixels (say, 100 instead of 9) and fits a smooth polynomial surface through all of them. The derivative of that smooth surface is our edge estimate. More pixels means more noise averaging, and the polynomial fit means we are not blurring edges the way Gaussian smoothing does.

2.3 Why Direction Matters

An edge has a direction. A vertical edge has a strong horizontal derivative and a weak vertical one. A diagonal edge has strong derivatives in neither the horizontal nor vertical direction individually, but a strong derivative perpendicular to the edge.

Rather than computing derivatives in just two directions (horizontal and vertical) and combining them, we try many candidate directions (say, 18 angles evenly spaced around the circle) and pick the one that gives the biggest response. This is what “anisotropic” means in the name: the filter looks different in different directions.

3 Step 2: The Point Filter (One Pixel, One Direction)

This is the foundation of everything. Before we worry about lines or stencils, we need to understand how to estimate a derivative at a single pixel in a single direction.

3.1 Setting Up Local Coordinates

Pick a target pixel at position (X_0, Y_0) in the image, where X_0 is its column (horizontal position) and Y_0 is its row (vertical position). Pick a candidate edge direction θ_k , which is just an angle measured in radians. If we are testing $N_s = 18$ directions, then θ_k would be one of 18 evenly spaced angles around the circle ($0^\circ, 20^\circ, 40^\circ, \dots, 340^\circ$). The subscript k is just an index that says “which angle are we currently testing” ($k = 0, 1, 2, \dots, 17$).

We set up a local coordinate system centered on our pixel, rotated so that x points along the normal to the edge (the direction we want the derivative in) and y points along the edge itself.

For any neighbor pixel at (X_i, Y_i) , where i is just an index labeling which neighbor we are talking about ($i = 1, 2, \dots, N_p$), its local coordinates are:

$$\begin{aligned} x_i &= (X_i - X_0) \cos \theta_k + (Y_i - Y_0) \sin \theta_k \\ y_i &= -(X_i - X_0) \sin \theta_k + (Y_i - Y_0) \cos \theta_k \end{aligned} \quad (1)$$

Here x_i is the position of neighbor i in the rotated coordinate system along the normal direction, and y_i is its position along the tangent direction. The terms $(X_i - X_0)$ and $(Y_i - Y_0)$ are just “how far is this neighbor from the target pixel in the original image coordinates.” The $\cos \theta_k$ and $\sin \theta_k$ terms rotate those offsets into the edge-aligned frame.

This is just a standard 2D rotation matrix applied to each neighbor’s offset. We are re-expressing each neighbor’s position relative to our target pixel, in a coordinate system aligned with the candidate edge direction.

To give a concrete example, suppose our target pixel is at $(50, 50)$ and we are testing $\theta_k = 0^\circ$ (a vertical edge). Then $\cos 0 = 1$ and $\sin 0 = 0$, so the rotation is trivial: $x_i = X_i - 50$ and $y_i = Y_i - 50$. For $\theta_k = 45^\circ$, the axes are rotated 45 degrees, so the x and y coordinates become mixtures of the horizontal and vertical offsets.

3.2 Fitting a Polynomial Surface

Now we model the image brightness near our target pixel as a polynomial in these local coordinates. Using a Taylor expansion of order d (we typically use $d = 4$). Here d is the polynomial degree, a design choice we make. Higher d means the polynomial can represent more complex shapes, but also means more unknowns to solve for. The function f is the image brightness: $f(x_i, y_i)$ means “the brightness at the position (x_i, y_i) .”

$$f(x_i, y_i) \approx \underbrace{f^0}_{\text{constant}} + \underbrace{f_x^0 x_i}_{\text{linear in } x} + \underbrace{f_y^0 y_i}_{\text{linear in } y} + \underbrace{\frac{f_{xx}^0}{2} x_i^2}_{\text{quadratic}} + \dots \quad (2)$$

Here is what every symbol means.

f^0 is the brightness value right at the target pixel (the constant term). f_x^0 is the first partial derivative of brightness with respect to x at the target pixel, i.e. how fast the brightness changes in the x -direction. This is the number we ultimately want because it tells us the edge strength. f_y^0 is the same thing but in the y -direction (along the edge, which we care less about). f_{xx}^0 is the second derivative with respect to x (how the rate of change itself is changing), and so on for all the higher-order terms. The superscript 0 on all of these just means “evaluated at the origin of our local coordinate system,” i.e. at the target pixel.

There are $M = (d + 1) \frac{d+2}{2}$ unknown coefficients in this expansion. This formula comes from counting how many distinct monomial terms exist in a 2D polynomial of degree d . For $d = 4$, that is $M = \frac{5 \times 6}{2} = 15$ unknowns. For $d = 2$, it would be $M = \frac{3 \times 4}{2} = 6$ unknowns.

The key point is that all of these unknowns ($f^0, f_x^0, f_y^0, f_{xx}^0, \dots$) are what we are trying to solve for. We know $f(x_i, y_i)$ at each neighbor (we read it from the image), and we know (x_i, y_i) (we computed it from the rotation). The unknowns are the derivatives. Once we solve for them, we grab f_x^0 and that is our edge strength at this pixel in this direction.

3.3 Gathering Neighbors

We select N_p neighbor pixels around our target. N_p stands for “number of points” and is a design parameter we choose. These neighbors are chosen as the N_p closest integer-coordinate pixels to the target, forming an approximately circular patch. For $N_p = 100$, this circle has a radius of about 6 pixels.

Each neighbor gives us one equation. For neighbor i , we know two things: its local coordinates (x_i, y_i) (computed from Equation 1) and its brightness value $f(X_i, Y_i)$ (read directly from the image). We plug (x_i, y_i) and $f(X_i, Y_i)$ into Equation 2. That gives us one equation where the left side is a known number (the brightness we read from the image) and the right side is an expression involving the 15 unknown derivatives multiplied by known coordinate values. One neighbor, one equation.

With $N_p = 100$ neighbors, we get 100 equations. With $M = 15$ unknowns, we have far more equations than unknowns. This is called an overdetermined system. There is no single solution that satisfies all 100 equations exactly (because of noise in the image), but we can find the “best fit” solution that comes closest to satisfying all of them. This best fit naturally averages out the noise, and the more equations we have relative to unknowns, the better the averaging.

3.4 The Linear System

We can write all N_p equations at once as a single matrix equation:

$$\mathbf{A}_{\theta_k} \mathbf{z} = \mathbf{b} \quad (3)$$

This is the compact way of writing “100 equations with 15 unknowns.” Here is what each piece is.

$\mathbf{A}_{\theta_k}$ is the design matrix, with dimensions $N_p \times M$ (e.g., 100×15). The subscript θ_k reminds us that this matrix depends on which angle we are testing, because the local coordinates (x_i, y_i) change when we rotate. Each row of $\mathbf{A}$ corresponds to one neighbor pixel i . The entries in that row are the monomial basis terms (the $1, x_i, y_i, \frac{x_i^2}{2}, \dots$ from Equation 2) evaluated at that neighbor’s local coordinates. For example, for a 2nd-order fit ($d = 2, M = 6$), row i would be $\left[1, x_i, y_i, \frac{x_i^2}{2}, \frac{y_i^2}{2}, x_i y_i\right]$. For $d = 4$ there are 15 such entries per row.

$\mathbf{z}$ is the vector of unknowns, with dimensions $M \times 1$ (e.g., 15×1). It contains all the derivative coefficients we are solving for: $\mathbf{z} = \left[f^0, f_x^0, f_y^0, \frac{f_{xx}^0}{2}, \dots\right]^\top$. These are the same unknowns from Equation 2. Once we solve for $\mathbf{z}$, we will grab its second entry (f_x^0) as our edge strength.

$\mathbf{b}$ is the data vector, with dimensions $N_p \times 1$ (e.g., 100×1). It contains the actual brightness values we read from the image at each neighbor location: $\mathbf{b} = \left[f(X_1, Y_1), f(X_2, Y_2), \dots, f(X_{N_p}, Y_{N_p})\right]^\top$. This is the only part that changes from pixel to pixel; $\mathbf{A}$ is the same for every pixel at a given angle θ_k .

The equation $\mathbf{A}\mathbf{z} = \mathbf{b}$ says: “if the polynomial model were perfect, then multiplying the design matrix by the unknown derivatives would reproduce the observed brightness values exactly.” In practice it will not be exact because of noise, so we find the best approximation.

3.5 Solving It: The Pseudoinverse

Since we have more equations than unknowns ($N_p > M$), there is no exact solution. We find the least-squares best fit, meaning the $\mathbf{z}$ that minimizes the total squared error across all 100 equations:

$$\hat{\mathbf{z}} = (\mathbf{A}_{\theta_k}^\top \mathbf{A}_{\theta_k})^{-1} \mathbf{A}_{\theta_k}^\top \mathbf{b} \quad (4)$$

Here $\hat{\mathbf{z}}$ is the estimated (best-fit) version of $\mathbf{z}$, the hat symbol $\hat{\cdot}$ is standard notation for “estimated.” The expression $\mathbf{A}_{\theta_k}^\top$ is the transpose of $\mathbf{A}$ (rows and columns swapped), and $(\cdot)^{-1}$ means matrix inverse. This formula is the standard least-squares solution from linear algebra; it is the value of $\mathbf{z}$ that minimizes $\|\mathbf{A}\mathbf{z} - \mathbf{b}\|^2$.

We can split this formula into two pieces. Define the pseudoinverse matrix:

$$\mathbf{P}_{\theta_k} = (\mathbf{A}_{\theta_k}^\top \mathbf{A}_{\theta_k})^{-1} \mathbf{A}_{\theta_k}^\top \quad (5)$$

$\mathbf{P}_{\theta_k}$ is called the Moore-Penrose pseudoinverse of $\mathbf{A}_{\theta_k}$. It is an $M \times N_p$ matrix (e.g., 15×100). The critical property of $\mathbf{P}$ is that it transforms a vector of N_p brightness values directly into the M fitted derivative coefficients. In other words, the solution is simply $\hat{\mathbf{z}} = \mathbf{P}_{\theta_k} \mathbf{b}$: one matrix-vector multiply and we have all 15 derivatives.

The reason we give $\mathbf{P}$ a name is that it depends only on the geometry (the neighbor positions and the angle θ_k), not on the image. So we can compute $\mathbf{P}_{\theta_k}$ once ahead of time and reuse it for every pixel.

3.6 Extracting the Derivative We Want

We do not actually need all 15 derivatives. We only need $\hat{f}_x^0$, the normal derivative (edge strength). This is the second entry of $\hat{\mathbf{z}}$ (index 1 if we start counting from 0, because index 0 is the constant f^0). Since $\hat{\mathbf{z}} = \mathbf{P}_{\theta_k} \mathbf{b}$, the second entry of $\hat{\mathbf{z}}$ is just the dot product of row 1 of $\mathbf{P}$ with $\mathbf{b}$:

$$\hat{f}_x^{(k)} = \mathbf{p}_{\text{fx}}^{(k)} \cdot \mathbf{b} \quad (6)$$

Here $\hat{f}_x^{(k)}$ is the estimated normal derivative at angle θ_k . The superscript (k) reminds us which angle we are testing. The vector $\mathbf{p}_{\text{fx}}^{(k)}$ is row 1 of the pseudoinverse matrix $\mathbf{P}_{\theta_k}$. It has N_p entries (one per neighbor pixel). The subscript “fx” stands for “ f_x -extraction,” meaning this is the row of $\mathbf{P}$ that extracts the f_x derivative. The vector $\mathbf{b}$ is the same brightness vector from before (the brightness values at all N_p neighbors).

This is the key insight of the entire derivation. The estimated derivative at one pixel in one direction is just a dot product: take each neighbor’s brightness, multiply it by a fixed weight from $\mathbf{p}_{\text{fx}}^{(k)}$, and add them all up. No matter how fancy the polynomial fitting sounds, in the end it is just “multiply each neighbor’s brightness by a weight and sum.”

The weights $\mathbf{p}_{\text{fx}}^{(k)}$ depend on the geometry (where the neighbors are and what angle θ_k we are testing) but not on the image. They can be computed once in advance and reused for every pixel in the image.

3.7 Picking the Best Direction

We repeat this for all N_s candidate angles and pick the direction with the strongest response. The angles are evenly spaced: $\theta_k = k \cdot 2\frac{\pi}{N_s}$ for $k = 0, 1, \dots, N_s - 1$. Here N_s is the number of orientations we test (a design choice; we use $N_s = 18$, giving angles every 20°).

$$k^* = \arg \max_k |\hat{f}_x^{(k)}| \quad (7)$$

Here k^* is the index of the winning orientation, the one whose absolute derivative response $|\hat{f}_x^{(k)}|$ is largest. The operator $\arg \max_k$ means “find the value of k that maximizes the expression.” The absolute value $|\cdot|$ is needed because an edge could produce a positive or negative derivative depending on which side is brighter; we care about the magnitude, not the sign.

The gradient magnitude (edge strength) at this pixel is $|\hat{f}_x^{(k^*)}|$, and the edge angle is θ_{k^*} . This completes the point filter.

4 Step 3: The Line Extension (Making It Better, But Slower)

The point filter works, but it only looks at one circular patch around the target pixel. For low-contrast edges or noisy images, one patch might not give a strong enough signal. The line filter fixes this by looking at multiple patches arranged along the candidate edge direction.

4.1 Virtual Evaluation Points

Instead of fitting the polynomial only at the target pixel (X_0, Y_0) , we also fit it at $(2m + 1)$ virtual positions spaced along the edge direction. Here m is the half-width of the line, a design parameter we choose. The term $(2m + 1)$ counts the total number of positions: m on each side of center, plus the center itself.

$$(X_j, Y_j) = (X_0 + j \cos \theta_k, Y_0 + j \sin \theta_k), \quad j \in \{-m, \dots, 0, \dots, m\} \quad (8)$$

Here (X_j, Y_j) is the position of the j -th virtual evaluation point. The index j runs from $-m$ to $+m$ and labels each virtual position along the line. When $j = 0$, we are at the original target pixel. When $j = 1$, we have moved one step in the direction θ_k . When $j = -1$, we have moved one step in the opposite direction. The $\cos \theta_k$ and $\sin \theta_k$ terms convert the step along the angle θ_k into horizontal and vertical pixel offsets.

For $m = 7$, this gives $2(7) + 1 = 15$ positions (from $j = -7$ to $j = 7$). At each one, we run the entire point filter from Step 2: gather N_p neighbors around (X_j, Y_j) , build the design matrix, solve via pseudoinverse, extract $\hat{f}_x^{(j,k)}$. The notation $\hat{f}_x^{(j,k)}$ now has two superscripts: j tells us which virtual position, and k tells us which angle.

Think of it this way. We are sliding a circular patch along the candidate edge direction and measuring the edge strength at each position. If there really is an edge running in that direction, all 15 patches should see it, and the combined evidence is much stronger than any single patch.

4.2 Combining the Estimates

We combine the $(2m + 1)$ derivative estimates using a Gaussian-weighted average:

$$R_k = \sum_{j=-m}^m w_j \cdot \hat{f}_x^{(j,k)} \quad (9)$$

Here R_k is the line filter response for orientation k , a single number that summarizes how strong the edge evidence is in direction θ_k when we consider all the virtual positions together. The summation $\sum_{j=-m}^m$ means “add up for every virtual position j from $-m$ to m .” Each term in the sum is the Gaussian weight w_j times the derivative estimate $\hat{f}_x^{(j,k)}$ at virtual position j .

The weights w_j are defined as:

$$w_j = \exp\left(-\frac{j^2}{2\sigma^2}\right), \quad \sigma = \frac{m}{2} \quad (10)$$

Here w_j is the Gaussian weight for virtual position j . It is just a bell-curve value: the $\exp$ function (exponential, e raised to a power) produces a number between 0 and 1. When $j = 0$ (center position), $w_0 = \exp(0) = 1$ (maximum weight). As $|j|$ increases (positions farther from center), w_j decreases

toward zero. The parameter σ controls how fast the weights decay; we set $\sigma = \frac{m}{2}$, which means positions beyond about $\frac{m}{2}$ steps from center contribute very little.

The Gaussian weighting means positions near the center (small $|j|$) count more than positions far away (large $|j|$). This is sensible: the target pixel is at $j = 0$, and positions farther away are less relevant.

For $m = 7$ ($\sigma = 3.5$), the weights look like this: $w_0 = 1.0$ (center), $w_{\pm 1} \approx 0.96$, $w_{\pm 3} \approx 0.68$, $w_{\pm 7} \approx 0.05$ (small but nonzero at the ends).

4.3 Why This Is Expensive

Every one of those 15 virtual positions requires its own gather of N_p pixels, its own matrix-vector product with the pseudoinverse, and its own extraction of the derivative. With $N_p = 100$ and $m = 7$:

- $15 \times 100 = 1500$ pixel reads per direction
- 15 separate matrix-vector multiplies
- All repeated for $N_s = 18$ directions

That is $15 \times 100 \times 18 = 27,000$ pixel reads per target pixel, plus thousands of multiply-add operations. For a 1280 x 720 image, that is about 25 billion pixel reads. This is why the naive LF is slow and uses tons of memory.

5 Step 4: The Algebraic Collapse (The Key Step)

Here is where the magic happens. We are going to show that the entire line filter computation, all 15 polynomial fits combined with Gaussian weighting, is equivalent to a single weighted sum over pixel values. No matrices, no pseudoinverses, no loops at runtime.

5.1 Expanding the Line Response

Let us write out what R_k actually computes, in full detail. We start with the line response from Equation 9:

$$R_k = \sum_{j=-m}^m w_j \cdot \hat{f}_x^{(j,k)} \quad (11)$$

This says: “for each virtual position j , take the derivative estimate $\hat{f}_x^{(j,k)}$, multiply it by the Gaussian weight w_j , and add them all up.”

Now we substitute what $\hat{f}_x^{(j,k)}$ actually is. From Equation 6, we know that the derivative estimate at any single position is a dot product of the pseudoinverse row with the brightness values. At virtual position j :

$$R_k = \sum_{j=-m}^m w_j \cdot \left(\mathbf{p}_{\text{fx}}^{(k)} \cdot \mathbf{b}_j \right) \quad (12)$$

Here $\mathbf{b}_j$ is the vector of N_p brightness values gathered around virtual position j (not around the original target pixel, but around the shifted position (X_j, Y_j)). The weight vector $\mathbf{p}_{\text{fx}}^{(k)}$ is the same for every virtual position because the neighbor pattern and the polynomial model are the same; only the center of the patch moves.

Now we expand the dot product $\mathbf{p}_{\text{fx}}^{(k)} \cdot \mathbf{b}_j$ into its individual terms:

$$R_k = \sum_{j=-m}^m w_j \sum_{i=1}^{N_p} p_i^{(k)} \cdot f(X_j + \Delta x_i, Y_j + \Delta y_i) \quad (13)$$

Here $p_i^{(k)}$ is the i -th entry of $\mathbf{p}_{\text{fx}}^{(k)}$, i.e., the pseudoinverse weight for the i -th neighbor. The term $(\Delta x_i, \Delta y_i)$ is the offset of neighbor i relative to the center of its patch (these are the same offsets

for every virtual position, because we always use the same circular neighbor pattern). The function $f(X_j + \Delta x_i, Y_j + \Delta y_i)$ is just “read the brightness from the image at the pixel located at the virtual position plus the neighbor offset.”

The inner sum $\sum_{i=1}^{N_p}$ loops over all N_p neighbors. The outer sum $\sum_{j=-m}^m$ loops over all $(2m+1)$ virtual positions. Together, they produce $(2m+1) \times N_p$ terms.

Now substitute the definition of (X_j, Y_j) from Equation 8, which is $(X_0 + j \cos \theta_k, Y_0 + j \sin \theta_k)$:

$$R_k = \sum_{j=-m}^m \sum_{i=1}^{N_p} \underbrace{w_j \cdot p_i^{(k)}}_{\text{a single number}} \cdot f \left(\underbrace{X_0 + j \cos \theta_k + \Delta x_i}_{\text{x-coordinate of pixel}}, \underbrace{Y_0 + j \sin \theta_k + \Delta y_i}_{\text{y-coordinate of pixel}} \right) \quad (14)$$

Let us pause and stare at this equation. It says: R_k is a sum of $(2m+1) \times N_p$ terms. Each term has two parts. Part one is $w_j \cdot p_i^{(k)}$, which is a product of two known numbers: the Gaussian weight at position j and the pseudoinverse weight for neighbor i . Both of these are determined by the filter geometry and can be computed before we ever look at the image. Part two is $f(\dots)$, which is a single pixel brightness value read from the image at a specific location.

So each term is just “a precomputable number times a pixel value.” And R_k is the sum of all such terms.

That is it. The entire line filter, the polynomial fitting, the pseudoinverse, the Gaussian weighting, all of it, reduces to “multiply some pixels by some precomputed numbers and add up.” This is the algebraic collapse.

5.2 Why Does This Work?

It works because every operation in the pipeline is linear:

1. Gathering neighbor pixels is linear (just selecting values from the image).
2. The matrix-vector product $\mathbf{P}_{\theta_k} \mathbf{b}$ is linear in $\mathbf{b}$.
3. Extracting one row of the result is linear.
4. Multiplying by w_j is linear.
5. Summing over j is linear.

When you compose linear operations, the result is linear. A linear operation on pixel values is always expressible as “multiply each pixel by a weight and sum.” There is no escaping this.

To make it even more concrete: imagine the entire image is a single vector $\mathbf{I}$ of all pixel values. Then R_k is some vector $\boldsymbol{\alpha}_k$ dotted with $\mathbf{I}$. The vector $\boldsymbol{\alpha}_k$ has a nonzero entry for every pixel that participates in the computation, and the value of that entry is the total weight that pixel receives. Most entries of $\boldsymbol{\alpha}_k$ are zero (pixels far from the target do not participate), so $\boldsymbol{\alpha}_k$ is sparse. But it is still just a dot product.

6 Step 5: Building the Fused Stencil

We have shown that R_k is a weighted sum over pixel values. But Equation 14 has $(2m+1) \times N_p$ terms. Many of those terms access the same pixel, because the circular neighborhoods of adjacent virtual positions overlap. We need to merge those duplicates.

6.1 The Overlap Problem (With Numbers)

Consider $m = 7$ and $N_p = 100$. Each of the 15 virtual positions gathers 100 neighbors from an approximately circular patch of radius ≈ 6 pixels. Adjacent virtual positions are only 1 pixel apart. So two adjacent patches share most of their pixels.

Concretely, virtual position $j = 0$ gathers the 100 pixels closest to (X_0, Y_0) . Virtual position $j = 1$ (at $\theta_k = 0^\circ$, this is one pixel to the right) gathers the 100 pixels closest to $(X_0 + 1, Y_0)$. These two circles of radius 6, centered 1 pixel apart, overlap in about 90 of their 100 pixels.

The raw stencil has $15 \times 100 = 1500$ entries, but many of those entries point to the same pixel.

6.2 Deduplication

We group all (j, i) pairs that map to the same integer pixel position and sum their weights:

$$\alpha_{k,\ell} = \sum_{\{(j,i): \text{round}(j \cos \theta_k + \Delta x_i, j \sin \theta_k + \Delta y_i) = \text{position } \ell\}} w_j \cdot p_i^{(k)} \quad (15)$$

Here $\alpha_{k,\ell}$ is the fused weight for orientation k at unique pixel position ℓ . The subscript ℓ is a new index that labels unique pixel positions in the fused stencil ($\ell = 1, 2, \dots, N'_k$). For each unique position ℓ , we find all the (j, i) pairs from the raw stencil that land on that same pixel (after rounding to integer coordinates), and we add up their individual weights $w_j \cdot p_i^{(k)}$. The “round” operation reflects the fact that the computed offsets $j \cos \theta_k + \Delta x_i$ may not be exact integers, so we snap them to the nearest pixel.

After deduplication, the stencil shrinks dramatically. For $m = 7$, $N_p = 100$: from 1500 raw entries down to about 264 unique pixel positions. That is an 82% reduction.

The fused stencil for orientation k is then just two arrays. The first array stores the pixel offsets $(\tilde{\delta}_\ell^x, \tilde{\delta}_\ell^y)$, which say “how far from the target pixel is stencil position ℓ , in the x and y directions.” The tilde ($\tilde{\cdot}$) distinguishes these from the raw offsets; these are the deduplicated, unique positions. The second array stores the corresponding fused weights $\alpha_{k,\ell}$. Together, these two arrays completely define the filter for orientation k . There are N'_k entries, where N'_k is the number of unique positions (e.g., 264 instead of 1500).

6.3 The Final Formula

After deduplication, the line filter response is:

$$R_k = \sum_{\ell=1}^{N'_k} \alpha_{k,\ell} \cdot f(X_0 + \tilde{\delta}_\ell^x, Y_0 + \tilde{\delta}_\ell^y) \quad (16)$$

This says: “for each of the N'_k unique stencil positions, read the pixel brightness at that position (f evaluated at the target pixel plus the offset), multiply by its fused weight $\alpha_{k,\ell}$, and sum everything up.” That is the entire filter. One loop, one multiply-add per stencil position, done.

Or in vector notation:

$$R_k = \alpha_k^\top \mathbf{g}_k \quad (17)$$

Here $\alpha_k \in \mathbb{R}^{N'_k}$ is the precomputed weight vector (all N'_k fused weights stacked into a column), and $\mathbf{g}_k \in \mathbb{R}^{N'_k}$ is the gathered intensity vector (the brightness values read from the image at the N'_k stencil positions). The notation $\alpha_k^\top$ means the transpose of α_k (turning a column into a row so the dot product works). The notation $\mathbb{R}^{N'_k}$ just means “a vector of N'_k real numbers.”

This is mathematically identical to Equation 14. The same pixels participate. The same total weights are applied. The same answer comes out. We have just reorganized the computation to avoid redundant work.

7 Step 6: Why This Is So Much Faster

7.1 The Naive LF (Before Fusion)

For each pixel, for each of N_s orientations, the naive LF does:

1. Loop over $(2m + 1)$ virtual positions.
2. At each virtual position, gather N_p neighbors from the image (a scattered memory read).
3. Compute $\mathbf{P}_{\theta_k} \mathbf{b}_j$ (a matrix-vector product: $M \times N_p$ multiplies).
4. Extract the normal derivative (pick one element).
5. Multiply by w_j .
6. Sum the $(2m + 1)$ weighted derivatives.

Total work per pixel per orientation: $(2m + 1) \times N_p$ pixel reads, plus $(2m + 1)$ matrix-vector products of size $M \times N_p$.

Total pixel reads per image pixel: $(2m + 1) \times N_p \times N_s$. For $m = 7, N_p = 100, N_s = 18$: $15 \times 100 \times 18 = 27,000$ reads. Plus storing all those intermediate values in GPU memory.

7.2 The Fused ASF (After Fusion)

For each pixel, for each of N_s orientations, the ASF does:

1. Gather N'_k pixel values from the image (one set of scattered reads).
2. Multiply each by its precomputed weight and sum (one dot product).

Total pixel reads per image pixel: $N'_k \times N_s$. For $m = 7, N_p = 100, N_s = 18$: $264 \times 18 = 4,752$ reads. That is 5.7 times fewer reads, and zero matrix operations.

But the speedup is even larger than 5.7 times because:

- No matrix-vector products at runtime (those are baked into the weights).
- No intermediate storage (the naive approach stores huge tensors; the ASF stores only the input image and the output).
- The GPU kernel is a simple loop with predictable memory access, which modern hardware loves.

In practice, we measured 18 to 24 times speedup and 311 times reduction in GPU memory.

8 Step 7: A Worked Example

Let us trace through the entire process with tiny numbers to make it concrete. We will use $d = 1$ (linear polynomial, 3 unknowns), $N_p = 4$ (4 neighbors), $m = 1$ (3 virtual positions), $N_s = 1$ (one orientation, $\theta = 0$, vertical edge).

8.1 The Point Filter Weights

With $d = 1$ and $\theta = 0^\circ$, the polynomial model is $f(x, y) \approx f^0 + f_x^0 x + f_y^0 y$.

Suppose our 4 neighbors (relative to the center) are at offsets:

$$(\Delta x, \Delta y) \in \{(-1, 0), (1, 0), (0, -1), (0, 1)\} \quad (18)$$

The design matrix is:

$$\mathbf{A} = \begin{pmatrix} 1 & -1 & 0 \\ 1 & 1 & 0 \\ 1 & 0 & -1 \\ 1 & 0 & 1 \end{pmatrix} \quad (19)$$

The pseudoinverse (which you can verify by computing $(\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top$) is:

$$\mathbf{P} = \begin{pmatrix} \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ -\frac{1}{2} & \frac{1}{2} & 0 & 0 \\ 0 & 0 & -\frac{1}{2} & \frac{1}{2} \end{pmatrix} \quad (20)$$

Row 1 (the one we care about) is $\mathbf{p}_{\text{fx}} = [-1/2, 1/2, 0, 0]$.

This makes perfect sense. To estimate the horizontal derivative, the polynomial fit says “take the right neighbor minus the left neighbor, divided by 2.” The top and bottom neighbors get zero weight because they carry no information about the horizontal derivative. This is reassuringly intuitive.

8.2 The Line Extension

With $m = 1$, we have 3 virtual positions at $j \in \{-1, 0, 1\}$. Since $\theta = 0^\circ$, these are at horizontal offsets $(-1, 0), (0, 0), (1, 0)$.

Gaussian weights with $\sigma = \frac{m}{2} = 0.5$:

$$w_{-1} = \exp\left(-\frac{1}{2 \cdot 0.25}\right) = \exp(-2) \approx 0.135 \quad (21)$$

$$w_0 = \exp(0) = 1.0 \quad (22)$$

$$w_1 = \exp(-2) \approx 0.135 \quad (23)$$

8.3 The Raw Stencil

For each virtual position j , the 4 neighbors (at offsets from that virtual position) map to these absolute offsets from the target pixel:

- $j = -1$ (virtual position at $(-1, 0)$): neighbors at $(-2, 0), (0, 0), (-1, -1), (-1, 1)$
- $j = 0$ (virtual position at $(0, 0)$): neighbors at $(-1, 0), (1, 0), (0, -1), (0, 1)$
- $j = 1$ (virtual position at $(1, 0)$): neighbors at $(0, 0), (2, 0), (1, -1), (1, 1)$

The raw weight for each entry is $w_j \cdot p_i$:

j	i	Offset	w_j	p_i	$w_j \cdot p_i$
-1	1	(-2, 0)	0.135	$-\frac{1}{2}$	-0.068
-1	2	(0, 0)	0.135	$\frac{1}{2}$	+0.068
-1	3	(-1, -1)	0.135	0	0
-1	4	(-1, 1)	0.135	0	0
0	1	(-1, 0)	1.0	$-\frac{1}{2}$	-0.500
0	2	(1, 0)	1.0	$\frac{1}{2}$	+0.500
0	3	(0, -1)	1.0	0	0
0	4	(0, 1)	1.0	0	0
1	1	(0, 0)	0.135	$-\frac{1}{2}$	-0.068
1	2	(2, 0)	0.135	$\frac{1}{2}$	+0.068
1	3	(1, -1)	0.135	0	0
1	4	(1, 1)	0.135	0	0

Table 1: Raw stencil entries. 12 total (3 virtual positions times 4 neighbors).

8.4 Deduplication

Some offsets appear more than once. Specifically, $(0, 0)$ appears at $(j = -1, i = 2)$ with weight +0.068 and at $(j = 1, i = 1)$ with weight -0.068. These sum to zero.

Also, many entries have weight zero (the neighbors with $p_i = 0$), so we can drop them.

After deduplication, the fused stencil is:

Offset	Fused weight α
(-2, 0)	-0.068
(-1, 0)	-0.500
(0, 0)	0.000 (drops out)
(1, 0)	+0.500
(2, 0)	+0.068

Table 2: Fused stencil after deduplication. Reduced from 12 entries to 4 nonzero entries.

The result: $R_0 = -0.068 \cdot f(-2, 0) - 0.500 \cdot f(-1, 0) + 0.500 \cdot f(1, 0) + 0.068 \cdot f(2, 0)$.

This is a weighted central difference that emphasizes the immediate neighbors (weight 0.5) and lightly includes the next-nearest neighbors (weight 0.068). It is like a slightly wider, smarter version of the Sobel filter, computed automatically from the polynomial fitting + line extension math. And we can compute this weighted sum directly without ever building a matrix or solving a system.

9 Step 8: The General Case

The worked example used tiny numbers. In the real filter with $N_p = 100$, $m = 7$, and $d = 4$, the process is identical but the numbers are bigger.

The raw stencil has $(2m + 1) \times N_p = 15 \times 100 = 1500$ entries. After deduplication to unique pixel positions, about 264 entries remain. Each entry has a precomputed weight $\alpha_{k,\ell}$ that encodes the combined effect of the polynomial fitting, the pseudoinverse extraction, and the Gaussian line weighting.

The stencil shape is an elongated, anisotropic pattern that stretches along the candidate edge direction. When you plot the weights, you see a dipole pattern: positive weights on one side of the expected edge, negative weights on the other. This makes sense because the derivative should respond to the difference in brightness across the edge.

At runtime, for each pixel, the GPU kernel loops over the N_s orientations, and for each one, it gathers the pixel values at the stencil offsets, multiplies by the precomputed weights, and sums. The orientation with the largest absolute response wins.

10 Summary

The derivation has four conceptual steps.

The first step is recognizing that the point filter (polynomial fit followed by derivative extraction) is a linear operation on pixel values. The derivative at a single point is just $\mathbf{p}_{\text{fx}}^{(k)} \cdot \mathbf{b}$, a dot product of fixed weights with brightness values.

The second step is recognizing that the line extension (evaluating the point filter at multiple virtual positions and taking a Gaussian-weighted sum) is also linear. You are summing dot products, which is itself a dot product.

The third step is expanding the double sum (over virtual positions j and neighbors i) to get a single sum over (j, i) pairs, each pair contributing $w_j \cdot \mathbf{p}_i^{(k)}$ times one pixel value.

The fourth step is deduplication: grouping all (j, i) pairs that reference the same pixel, summing their weights, and storing only the unique positions and fused weights. This compressed stencil is the ASF.

The result is mathematically identical to the original line filter. Same pixels, same weights, same answer. But the computation goes from “build matrices, solve systems, loop over virtual positions”

to “gather pixels, multiply by precomputed weights, sum.” This is why the ASF is 18 to 24 times faster and uses 311 times less memory than the naive LF implementation.